

Embedded Benchmarking of Neural Network vs. Lookup Table-Based Current Reference Generation for PMSM Drives

1st Ebru Avci
Electrical Drives and Actuators
FEAAM GmbH
Munich, Germany
e.avci@feaam.de

2nd Johann S. Mayer
Electrical Drives and Actuators
University of the Bundeswehr Munich
Munich, Germany
johann.mayer@unibw.de

3rd Dieter Gerling
Electrical Drives and Actuators
University of the Bundeswehr Munich
Munich, Germany
dieter.gerling@unibw.de

Abstract—This paper presents an embedded benchmarking comparison between Neural Network (NN)- and Lookup Table (LUT)-based current reference generation for Permanent Magnet Synchronous Machine (PMSM) drives using Field-Oriented Control (FOC). Conventional control strategies such as Maximum Torque Per Ampere (MTPA) and Maximum Torque Per Voltage (MTPV) are often implemented using high-dimensional LUTs, which become inefficient as input dimensionality increases. As a scalable alternative, this work investigates an NN to replace these traditional methods. The NN was trained on loss-optimal reference data generated from constrained minimization. Both the NN and a corresponding 3D-LUT were implemented in C and deployed on an STM32H7B3I-DK microcontroller. Benchmarking shows that the NN offers a 97.2% reduction in memory usage while providing comparable accuracy to the LUT. The NN inference time of 0.5 ms comfortably meets the 2 ms real-time constraint of our application and falls well within the typical timing budget of a higher-level control loop. These results highlight the potential of NN-based control as an efficient and lightweight alternative to traditional methods for electric traction drives.

Index Terms—Permanent Magnet Synchronous Machine (PMSM), Field-Oriented Control (FOC), Neural Networks (NN), Lookup Tables (LUT), Embedded Systems, Efficiency Optimization, STM32.

I. INTRODUCTION

Permanent Magnet Synchronous Machines (PMSMs) are widely used in electric traction applications due to their high power density and efficiency. Field-Oriented Control (FOC) is the standard strategy for operating PMSMs, where torque generation is achieved by controlling the dq-axis current components i_d and i_q . To operate efficiently across different speeds and load conditions, conventional strategies such as Maximum Torque per Ampere (MTPA) and Maximum Torque per Voltage (MTPV) are typically employed to generate optimal current reference values (i_d^* , i_q^*). These strategies are often implemented using Lookup Tables (LUTs) or analytical expressions [1]. While effective, these methods are static and rely on precomputed surfaces. Their reliance on high-resolution sampling makes them susceptible to interpolation inaccuracies and sparse sampling, especially in high-dimensional input spaces. In embedded systems, LUT-based

approaches further encounter memory limitations as input dimensionality increases [2], [3]. Common 3D-LUTs, based on grids of torque, speed, and DC-link voltage (T^* , n , v_{DC}), are often reduced to 2D structures over (T^* , n) by introducing analytical approximations to the third input, which further limits accuracy [4], [5]. In practice, extending LUTs beyond three dimensions, for example, to incorporate temperature or magnetic flux variations, becomes impractical for microcontrollers with limited memory resources.

This motivates the investigation of Neural Networks as a compact and scalable alternative to the conventional MTPA/MTPV logic. While they are similarly static in deployment, NNs can interpolate high-dimensional nonlinear functions using relatively few stored parameters. This makes them well suited for embedded control functions where input dimensionality is high. Furthermore, NNs offer advantages in terms of adaptability and memory efficiency, which are crucial under strict memory constraints. Unlike LUTs, where memory demand grows exponentially with each added input dimension, the complexity of NN-based methods increases only moderately, making them suitable for real-time embedded applications with higher-dimensional inputs.

In this paper, a manually implemented NN and a corresponding 3D-LUT were both deployed on an STM32H7B3I-

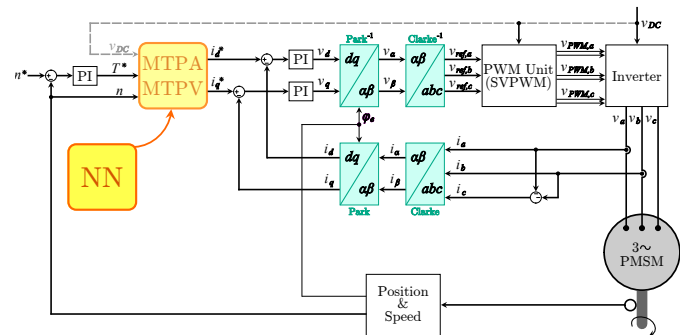


Fig. 1: Field-Oriented Control structure with NN replacing conventional MTPA/MTPV block

DK microcontroller and benchmarked against each other in terms of accuracy, execution time, and memory usage. The goal is to evaluate the viability of NN-based current reference generation for electric traction drives, particularly under memory constraints where conventional LUT-based methods become inefficient. This work addresses the need for more efficient and scalable control solutions in applications like electric vehicles, where optimizing energy usage and minimizing hardware resource requirements are critical. The integration of the NN into the existing FOC structure, replacing the conventional MTPA/MTPV logic block, is illustrated in Fig. 1.

II. DATA GENERATION VIA CONSTRAINED MINIMIZATION

The current reference values, i_d^* and i_q^* , are generated using a constrained minimization approach (CMA). For each operating point, defined by the motor speed n (rpm), desired torque T^* (Nm), and DC-link voltage v_{DC} (V), the objective is to minimize the total electrical losses while satisfying physical operating limits. The constrained optimization problem is formulated as:

$$\text{Minimize } P_L(i_d, i_q), \quad (1)$$

subject to:

$$\begin{aligned} T(i_d, i_q) &= T^* \\ i_d^2 + i_q^2 &\leq I_{\max}^2 \\ v_d^2 + v_q^2 &\leq V_{\max}^2 \end{aligned} \quad (2)$$

where $I_{\max}$ denotes the maximum allowable stator current, and $V_{\max} = v_{DC}/\sqrt{3}$ reflects the inverter's voltage limit [6], [7]. The stator voltages v_d and v_q are nonlinear functions of i_d , i_q , and n , which increases the complexity of the constrained optimization.

To reflect realistic machine behavior, the constrained minimization is executed over a dense 3D grid of input points (n, T^*, v_{DC}) using a physical machine model. This model, constructed from extensive testbench measurements, captures both magnetic nonlinearities and iron losses simultaneously, a key advantage over traditional flux map-based methods. It avoids the need for millions of difficult-to-obtain measurements required by purely data-driven approaches. Each grid point defines an operating condition for which the loss-optimal current references (i_d^*, i_q^*) are computed.

The resulting optimal current surfaces are highly nonlinear, piecewise-defined, and may exhibit discontinuities across transition zones. Fig. 2 shows the i_q^* surface as a representative example, as both i_d^* and i_q^* exhibit similar nonlinear characteristics. The generated dataset serves as a consistent reference for the NN training and the LUT construction, ensuring that both methods are evaluated on identical operating conditions. The constrained minimization is executed once per input combination in the (n, T^*, v_{DC}) grid, producing the optimal reference currents (i_d^*, i_q^*) that are stored and later reused for both training and benchmarking. The amount of training data depends directly on the resolution of the input grid. Finer grids yield more accurate reference surfaces but also increase dataset size and memory demand.

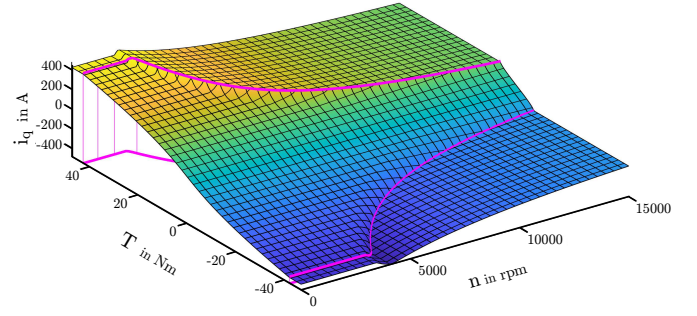


Fig. 2: Optimal current reference surface for i_q^* computed via constrained minimization

III. OVERALL NEURAL NETWORK ARCHITECTURE

The NN was designed to replace the conventional MTPA/MTPV logic block within the FOC structure by generating loss-optimal current references. As illustrated in Fig. 3a, the NN receives three-element input vector $\mathbf{x} = (n, T^*, v_{DC})$ and outputs the corresponding loss-optimal current references $\mathbf{y} = (i_d^*, i_q^*)$, which are then fed to the current controllers. The FOC current controllers then use these references to generate the desired torque while minimizing losses.

The task of the NN is to reconstruct the function:

$$(i_d^*, i_q^*) = f(n, T^*, v_{DC})$$

where $f(\cdot)$ is influenced by the system's physical constraints such as current and voltage limits. Because the mapping is static and the problem does not involve time-series data or spatial features, a fully connected, feedforward architecture was selected as a universal approximator [8]. Recurrent or convolutional layers are not required for this regression problem.

One representative network layer is shown in Fig. 3b. Each layer consists of a matrix multiplication, bias addition, and a non-linear activation function. The activation function was a scaled arctangent ($\arctan(\cdot)$), selected for its smooth, differentiable properties and numerical stability. Output de-normalization is applied after the final layer to recover the physical current values in amperes. The complete network architecture is illustrated in Fig. 4. It consists of approximately 100 neurons distributed across 12 fully connected layers,

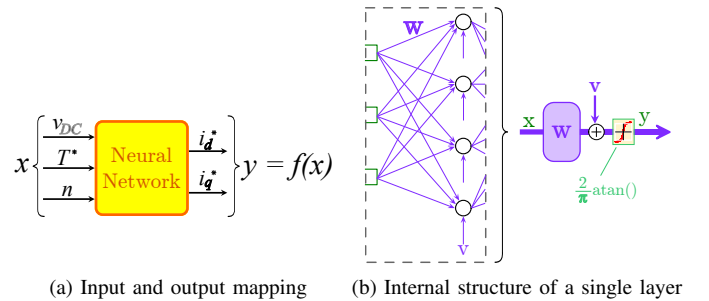


Fig. 3: NN structure: input-output mapping and internal layer

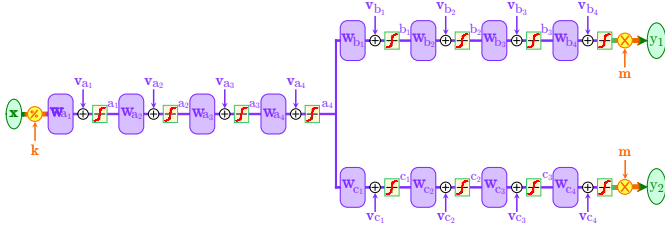


Fig. 4: NN architecture used for current reference generation

with around 1000 learned parameters (weights and biases). A double-headed output structure is used: after a shared core network, the output splits into two independent branches, each producing one scalar output (i_d^* and i_q^*). The NN model requires approximately 6228 multiply-accumulate (MAC) operations per inference. Training was performed offline using a dataset generated from constrained minimization, comprising one million training pairs and three million validation pairs. The optimizer used was the Levenberg–Marquardt algorithm to minimize the sum of squared errors (L_2 -norm). The number of iterations ranged between 3000 and 5000. Due to the highly overdetermined training setup (many more training pairs than network weights), the model is inherently strongly regularized. No explicit regularization techniques were required. The validation error generally converged toward the training error, and the final generalization error stabilized below 10^{-4} , demonstrating strong generalization capability.

The input to the network is a three-dimensional vector of floating-point values (n, T^*, v_{DC}), and the output consists of two scalar floating-point values (i_d^*, i_q^*). The connection between input and output is instantaneous, with no time delays or internal memory, making the model suitable for static inference. Inference was implemented using 32-bit floating-point arithmetic to ensure sufficient numerical precision. The network performs inference every 1–2 milliseconds, remaining well within the real-time constraints of typical high-level control loops.

The network was manually implemented in C to allow full control over the implementation, which was suitable for the moderate complexity of the model. Particular attention was given to CMSIS-DSP library compatibility, enabling optimized fixed-point inference on the STM32H7B3I-DK platform. No external machine learning libraries were used for embedded deployment.

IV. EMBEDDED DEPLOYMENT ON STM32H7B3I-DK

The STM32H7B3I-DK Discovery Kit was selected as the target platform for embedded benchmarking due to its performance and embedded relevance [9]. The development board is based on the STM32H7B3LI microcontroller, which features an ARM Cortex-M7 core operating at 280 MHz, supported by 2 MB of Flash memory and 1.4 MB of SRAM. The Cortex-M7 core includes a single-precision Floating Point Unit (FPU) and supports SIMD (Single Instruction Multiple Data) instructions through ARMv7E-M architecture extensions. This

allows accelerated execution of vector operations and basic matrix multiplications using CMSIS-DSP library [10]. However, the microcontroller is a single-core processor, and parallelism is only applied at the instruction level, not across multiple processing units. The system clock was configured to 280 MHz through STM32CubeIDE to achieve maximum performance. All implementations were developed manually in C, optimizing for fixed-point arithmetic to enhance execution speed and compatibility with CMSIS intrinsic functions. No external machine learning frameworks or dynamic memory allocations were used; all buffers were statically allocated during compilation to ensure deterministic memory behavior.

This platform was used for the full deployment and evaluation of both the manually implemented NN and the corresponding 3D-LUT, covering memory usage, execution time, and functional accuracy under real embedded conditions.

MATLAB Benchmark Overview

Before the deployment on embedded hardware, an initial benchmarking of NNs and LUTs was performed in MATLAB environment to evaluate the fundamental trade-offs between memory usage and computational speed. The benchmark utilized a simple synthetic test case: a 3-input, 1-output paraboloid function defined as:

$$V(x, y, z) = x^2 + y^2 + z^2$$

where x , y , and z represent generic input dimensions corresponding conceptually to operating parameters such as speed, torque demand, and DC-link voltage. This function was selected as a non-linear but well-defined test case to evaluate the interpolation and approximation capabilities of both methods.

For the LUT, a 3D grid with 201 sample points along each dimension was generated, resulting in approximately 8 million stored values. Trilinear interpolation was used to compute output values during evaluation. For the NN, a manually configured feedforward model with approximately 2000 weights and biases was trained to approximate the same function. Both models were benchmarked using 1,000,000 evaluations executed in double precision (64-bit) on a 16-core workstation (3.9 GHz). The benchmark was constrained to standard arithmetic operations and common math functions such as $\arctan(\cdot)$.

Based on memory allocation estimates, the LUT requires approximately 65 MB of memory, whereas the NN requires only 18.2 kB, highlighting the significant difference in memory footprint, especially when scaling to higher input dimensions. In terms of computational time, the LUT-based interpolation completed one million evaluations in approximately 9.6 seconds, while the NN required about 142 seconds. Thus, although LUTs provide faster evaluation speeds, they impose substantially greater memory demands compared to NN-based methods. The MATLAB profiler summary is shown in Fig. 5.

Embedded Implementation

After the preliminary comparison in MATLAB, the fully developed and trained NN and the corresponding 3D-LUT

Function Name	Calls	Total Time (s) *	Self Time* (s)	Total Time Plot (dark band = self time)
Testscript_NN_vs_LUT	1	154.522	3.092	
Testscript_NN_vs_LUT>Interferenc_NN	1000000	141.840	141.840	
Testscript_NN_vs_LUT>Interpolation_LUT_3D	1000000	9.571	9.571	
mainfield	1	0.019	0.019	
cmfield	1	0.001	0.001	

*Self time is the time spent in a function excluding any time spent in child functions. The time includes any overhead time resulting from the profiling process.

Fig. 5: MATLAB profiler output showing total computation times for NN and LUT benchmarks

were implemented on the STM32H7B3I-DK for actual embedded evaluation. Both implementations were manually coded in C. The NN model was implemented without relying on any external Artificial Intelligence (AI) or deep learning frameworks. The network architecture, as described previously, was translated into a sequence of matrix multiplications, bias additions, and arctangent activation function evaluations. Input normalization and output de-normalization steps were embedded into the processing chain. Particular attention was given to fixed-point readiness, and the code structure followed CMSIS-DSP library recommendations to allow later optimization if needed [11]. Similarly, the LUT was implemented manually as a statically allocated 3D array stored in Flash memory. A $100 \times 100 \times 25$ resolution was chosen as a compromise between accuracy and the available 2 MB of Flash memory. Trilinear interpolation was applied at runtime to compute the reference currents based on the eight nearest grid points surrounding each input query. All interpolation steps were manually coded using basic arithmetic operations to ensure deterministic and fast evaluation.

The system clock on the STM32 was configured to operate at 280 MHz. All computations were performed in single-precision floating-point format by default, with fixed-point preparation enabled where precision allowed. No dynamic memory allocation (malloc, free) was used; all arrays and buffers were allocated statically at compile time to ensure predictable memory behavior and real-time capability. The compiled memory usage was extracted using the `arm-none-eabi-size` utility. The resulting memory footprint was:

- NN implementation: approximately 56 kB of Flash memory.
- LUT implementation: approximately 2050 kB of Flash memory.

The lower memory usage of the embedded LUT compared to the 65 MB CMA-generated LUT used in MATLAB stems is a direct result of its reduced resolution, which enables deployment but increases interpolation error, particularly in highly nonlinear operating regions. This trade-off confirms the significant memory advantage of the NN approach, which

required no such compromise.

To validate functional accuracy and performance, a set of 20 identical input sets was created to compute the reference currents i_d^* and i_q^* using all three implementations: NN on STM32, LUT on STM32, and the CMA outputs (used as the reference). The outputs from both embedded implementations were then compared against the CMA reference values. Only five digits after the decimal point (up to 10^{-5}) were considered for comparison and visualization.

Fig. 6 summarizes the statistical accuracy across 20 test inputs using two key metrics: mean absolute error and maximum absolute error for both i_d^* and i_q^* . These metrics are computed for the STM32 NN and the STM32 LUT implementations with respect to the CMA reference. While all three methods yield similar current outputs, the statistical view provides clearer insight into the numerical fidelity of each approach. Table I further details point-wise relative errors for a subset of inputs. The results confirm that the NN outputs closely track the CMA reference, whereas the LUT exhibits more pronounced deviations due to its reduced resolution and associated interpolation errors in nonlinear operating regions. The average deviation for the STM32 NN outputs compared to the CMA

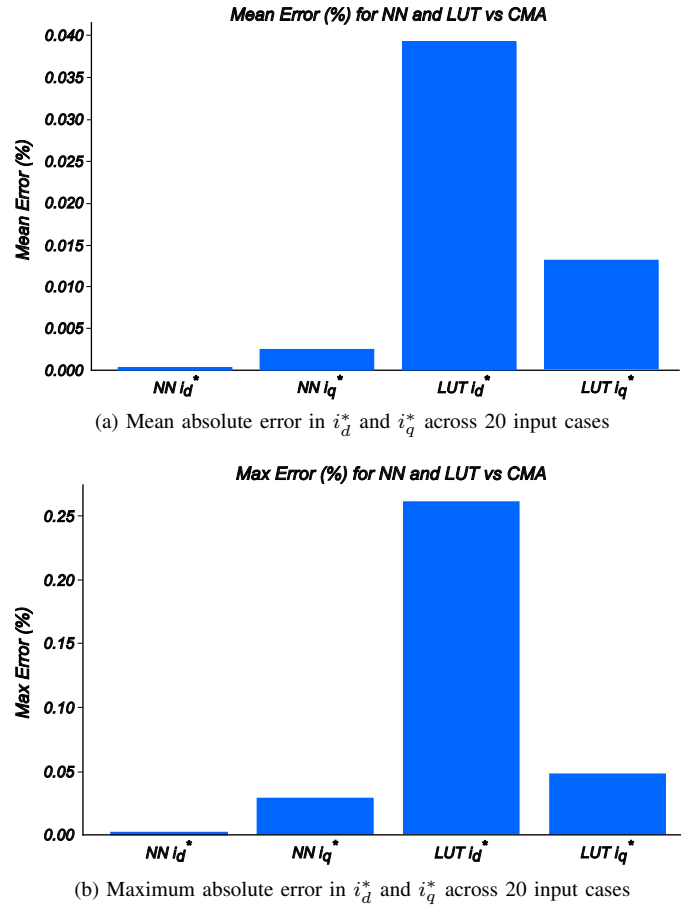


Fig. 6: Comparison of statistical accuracy metrics (mean and maximum absolute error) for NN and LUT implementations with respect to the CMA reference over 20 test inputs

TABLE I: Relative Error Comparison of STM32 NN and STM32 LUT Outputs Against CMA Reference

CMA Reference		STM32 NN		NN Relative Error (%)		STM32 LUT		LUT Relative Error (%)	
i_d^* (A)	i_q^* (A)	i_d^*	i_q^*	$RE_{i_d^*}$	$RE_{i_q^*}$	i_d^*	i_q^*	$RE_{i_d^*}$	$RE_{i_q^*}$
-75.11893	-957.38676	-75.11951	-957.38580	0.0008	0.0001	-75.28925	-957.36646	0.2267	0.0021
-57.03366	-457.48183	-57.03494	-457.48178	0.0022	0.0000	-57.18284	-457.41119	0.2616	0.0154
-96.47262	7.42215	-96.47227	7.42432	0.0004	0.0292	-96.53438	7.42073	0.0640	0.0191
-171.52119	467.44778	-171.52271	467.44708	0.0009	0.0002	-171.55690	467.48022	0.0208	0.0069
-260.40724	956.84783	-260.40738	956.84735	0.0001	0.0001	-260.40836	956.69653	0.0004	0.0158
-258.36401	-540.38671	-258.36575	-540.38446	0.0007	0.0004	-258.34012	-540.33020	0.0092	0.0105
-309.16226	-170.05707	-309.16263	-170.05623	0.0001	0.0005	-309.05746	-170.04651	0.0339	0.0062
-362.76823	189.60687	-362.76733	189.60802	0.0002	0.0006	-362.61328	189.57980	0.0428	0.0143
-415.98098	556.38248	-415.98172	556.38184	0.0002	0.0001	-415.98148	556.35724	0.0001	0.0045
-665.08504	834.71152	-665.08466	834.71118	0.0001	0.0000	-665.04749	834.62439	0.0057	0.0104

reference was below 0.06% for both i_d^* and i_q^* , whereas the LUT implementation showed significantly larger deviations of approximately 7.5% and 5.8%, respectively.

Fig. 7 shows the execution time measurements on STM32H7B3I for the same 20 input sets. The NN inference required approximately 0.5 ms per evaluation, whereas LUT interpolation was significantly faster at about 0.028 ms. Although the NN inference is computationally more demanding, it remains well within the system's real-time constraint of 2 ms per cycle. No CPU saturation or missed deadlines were observed during testing. These results demonstrate that NN-based current reference generation is not only memory-efficient but also capable of meeting real-time embedded requirements, with higher accuracy compared to LUT-based methods.

V. CONCLUSION

This paper presents an embedded benchmarking study of Neural Network (NN)- and Lookup Table (LUT)-based methods for current reference generation within a PMSM drive control. Both models were derived from the same constrained minimization dataset and deployed on the STM32H7B3I-DK microcontroller. The NN achieved a relative error below 0.06% for both current components, compared to higher relative errors of 5.8–7.5% for the LUT. Inference time for the NN

was 0.5 ms, well within the 2 ms real-time limit, though slower than the LUT's 0.028 ms. The NN required only 56 kB of Flash memory, compared to approximately 2 MB for the LUT, representing a reduction of over 97.2%. These results demonstrate that the NN-based approach offers a viable and scalable alternative for resource-constrained embedded systems, particularly when memory footprint or scaling to higher input dimensions challenge LUT-based methods. The compactness, adaptability, and generalization capability of the NN support its integration in next-generation electric traction drives, such as those used in electric vehicles where minimizing memory footprint and maximizing performance are essential.

ACKNOWLEDGMENT

This work has been developed as part of Eureka PENTA project 2021028 ECOMAI, funded by the German Federal Ministry of Research, Technology and Space (BMFTR) under grant number 16ME0567.

REFERENCES

- [1] M. Jaeger, S. Rick, and K. Hameyer, "Current simulation of a controlled pmsm including skew and torsional rotor vibrations," in *2018 XIII International Conference on Electrical Machines (ICEM)*, Alexandroupoli, Greece, 2018, pp. 111–117.
- [2] L. E. Bengtsson, "Lookup table optimization for sensor linearization in small embedded systems," *Journal of Sensor Technology*, vol. 2, no. 4, pp. 177–184, 2012. [Online]. Available: <https://www.scirp.org/journal/paperinformation?paperid=25530>
- [3] D.-Y. Kim and J.-H. Lee, "Compensation of interpolation error for look-up table-based pmsm control method in maximum power control," *Energies*, vol. 14, no. 17, p. 5526, 2021. [Online]. Available: <https://www.mdpi.com/1996-1073/14/17/5526>
- [4] E. Trancho, E. Ibarra, A. Arias, C. Salazar, I. Lopez, A. D. de Guereñu, and A. Peña, "Ipmsm torque control strategies based on luts and vct feedback for robust control under machine parameter variations," in *IECON 2016 - 42nd Annual Conference of the IEEE Industrial Electronics Society*, Florence, Italy, 2016, pp. 2833–2838.
- [5] D.-Y. Kim and J.-H. Lee, "Low cost simple look-up table-based pmsm drive considering dc-link voltage variation," *Energies*, vol. 13, no. 15, p. 3904, 2020.
- [6] Y. Jeong, S. Sul, S. Hiti, and K. Rahman, "Online minimum-copper-loss control of an interior permanent-magnet synchronous machine for automotive applications," *IEEE Transactions on Industry Applications*, vol. 42, no. 5, pp. 1222–1229, 2006.

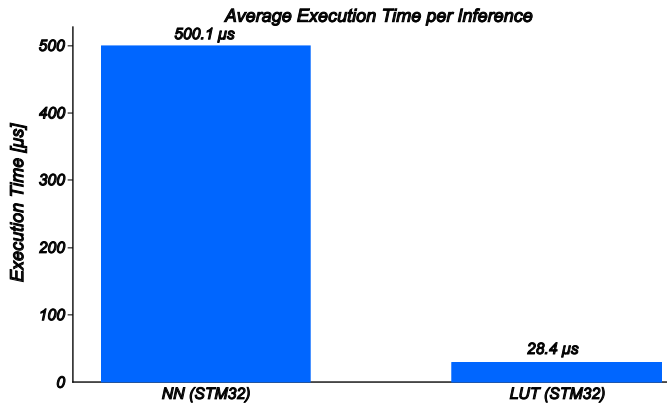


Fig. 7: Execution time per inference for NN and LUT implementations on STM32H7B3I-DK across 20 test inputs

- [7] R. Xiao, X. Yang, J. Li, and X. Jia, "Loss minimization control for interior permanent magnet synchronous motor in a wide speed range," in *2021 IEEE 4th International Electrical and Energy Conference (CIEEC)*, 2021.
- [8] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989. [Online]. Available: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- [9] STMicroelectronics, *UM2569 Discovery kit with STM32H7B3LI MCU*, STMicroelectronics, Jan 2025, revision 7.
- [10] L. Lai, N. Suda, and V. Chandra, "Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus," *arXiv:1801.06601*, 2018. [Online]. Available: <https://arxiv.org/abs/1801.06601>
- [11] Arm-software, n.d., accessed: 2024-12-16. [Online]. Available: <https://github.com/ARM-software/CMSIS-NN>